Day4 発展課題

goodwin_yoshida.py（時系列・相平面）

```
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

def goodwin_model( t , init , *param ):
    v = init[ 0 ]
    u = init[ 1 ]

    [ alpha , beta , gamma , rho , sigma ] = param

    dvdt = ( ( 1.0 / sigma - ( alpha + beta )  ) - u / sigma ) * v
    dudt = ( - ( alpha + gamma ) + rho * v ) * u

    return [ dvdt , dudt ]


init = [ 0.06 , 0.92 ]
t_span = [ 0 , 500 ]
t_eval = np.linspace( t_span[0] , t_span[1] , 10000 )

alpha = 0.02   # 技術進歩率
beta = 0.01    # 労働人口成長率
gamma = 0.01   # 賃金上昇の自然減衰項
rho = 0.5      # フィリップス曲線の感応度
sigma = 2.0    # 投資係数

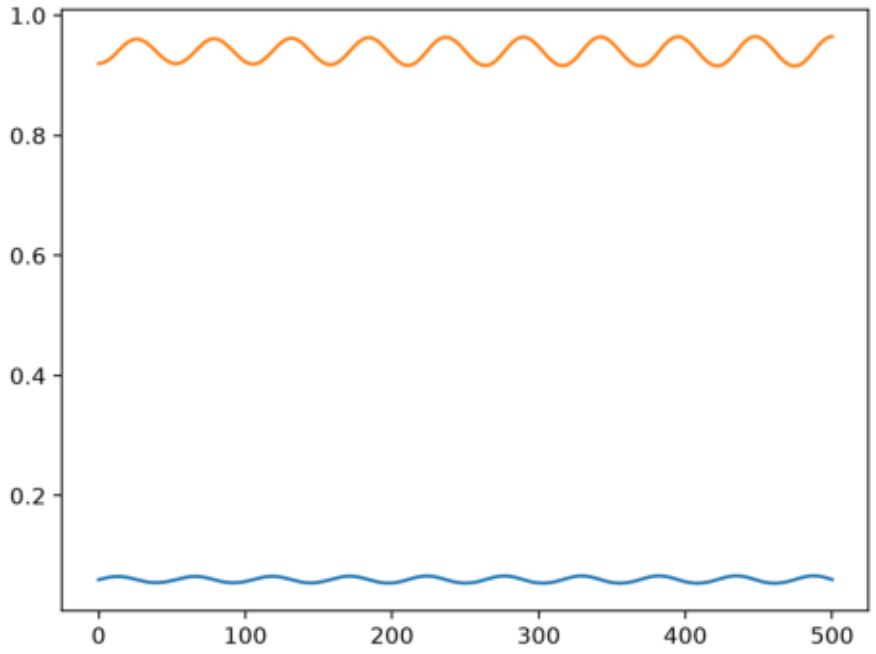
param = [ alpha , beta , gamma , rho , sigma ]

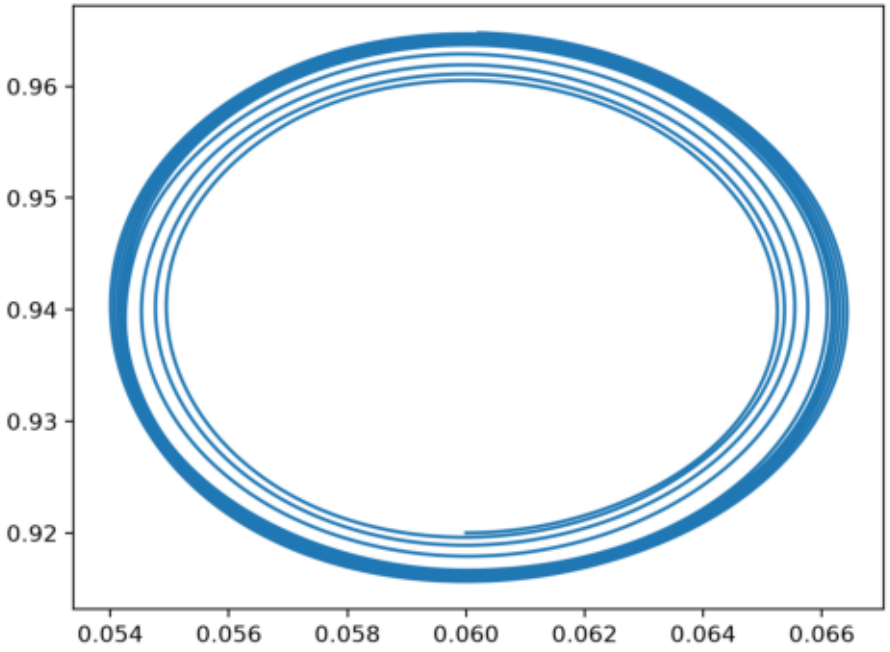
solution = solve_ivp( goodwin_model, t_span, init, t_eval=t_eval, method='RK45', args=param
        )

v , u = solution.y

plt.figure()
plt.plot( solution.t , v )
plt.plot( solution.t , u )
plt.show()

plt.figure()
plt.plot( v , u )
plt.show()
```


1.0
0.8
0.6
0.4
0.2
0
100
200
300
400
500


0.96
0.95
0.94
0.93
0.92
0.054
0.056
0.058
0.060
0.062
0.064
0.066

goodwin_overlay.py（ヌルクライン・ベクトル場・解軌跡）

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp

def goodwin_model(t, init, *param):
    v, u = init[0], init[1]

    [alpha, beta, gamma, rho, sigma] = param

    dvdt = ((1.0 / sigma - (alpha + beta)) - u / sigma) * v
    dudt = (-(alpha + gamma) + rho * v) * u

    return [dvdt, dudt]


init = [0.06, 0.92]
t_span = [0, 500]
t_eval = np.linspace(t_span[0], t_span[1], 10000)

alpha = 0.02   # 技術進歩率
beta = 0.01    # 労働人口成長率
gamma = 0.01   # 賃金上昇の自然減衰項
rho = 0.5      # フィリップス曲線の感応度
sigma = 2.0    # 投資係数

param = [alpha, beta, gamma, rho, sigma]

# Trajectory (tight tolerance: the default drifts outward by numerical error)
solution = solve_ivp(goodwin_model, t_span, init, t_eval=t_eval, method='RK45', args=param,
        rtol=1e-10, atol=1e-12)

v, u = solution.y

# Nullclines
v_null = (alpha + gamma) / rho        # du/dt = 0 : v = (α + γ) / ρ
u_null = 1 - sigma * (alpha + beta)   # dv/dt = 0 : u = 1 - σ(α + β)

# Vector field
N = 15
vg = np.linspace(0.05, 0.07, N)
ug = np.linspace(0.91, 0.97, N)

[V, U] = np.meshgrid(vg, ug)

dVdt = ((1.0 / sigma - (alpha + beta)) - U / sigma) * V
dUdt = (-(alpha + gamma) + rho * V) * U

plt.figure(figsize=(11, 4.5))

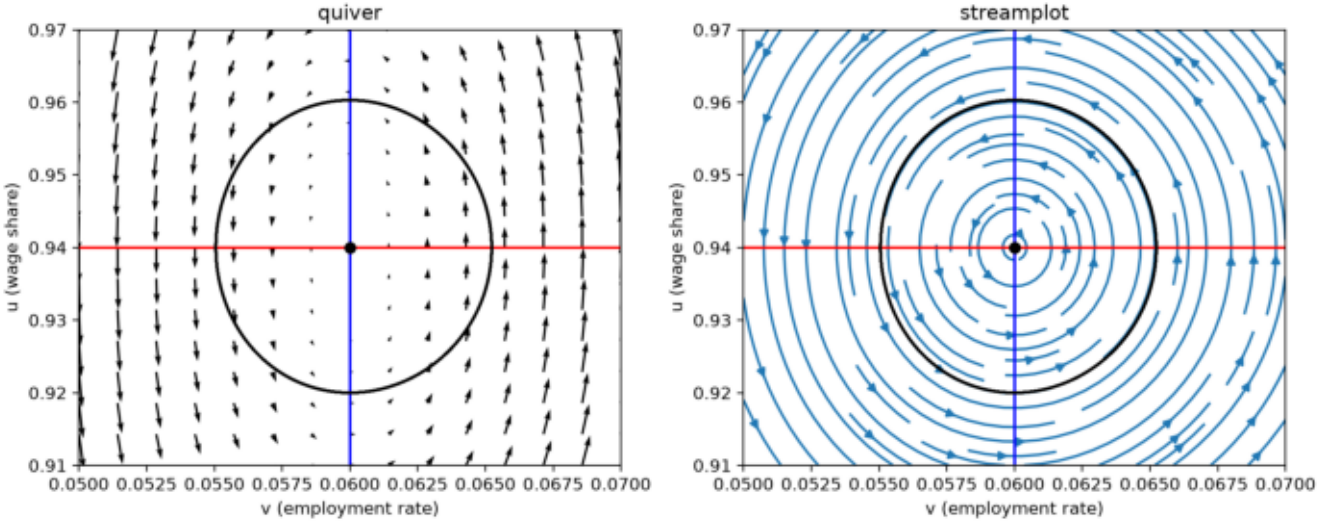
for i, method in enumerate(['quiver', 'streamplot']):
    plt.subplot(1, 2, i + 1)
    plt.title(method)

    if method == 'quiver':
        plt.quiver(V, U, dVdt, dUdt)
    else:
        plt.streamplot(V, U, dVdt, dUdt)
```

```
    plt.axvline(v_null, color='b')   # du/dt = 0
    plt.axhline(u_null, color='r')   # dv/dt = 0
    plt.plot(v, u, 'k-')
    plt.plot(v_null, u_null, 'ko')   # fixed point

    plt.xlim(vg[0], vg[-1])
    plt.ylim(ug[0], ug[-1])
    plt.xlabel('v (employment rate)')
    plt.ylabel('u (wage share)')

plt.tight_layout()
plt.show()
```



goodwin_jacobian.py（固定点・ヤコビ行列・固有値）

```
import sympy as sp

# 変数と定数（記号）
v, u = sp.symbols("v u", real=True)
alpha, beta, gamma, rho, sigma = sp.symbols("alpha beta gamma rho sigma", real=True,
        positive=True)

param_val = [ ( alpha , 0.02 ) , ( beta , 0.01 ) , ( gamma , 0.01 ) , ( rho , 0.5 ) , (
        sigma , 2.0 ) ]

f = ( ( 1 / sigma - ( alpha + beta ) ) - u / sigma ) * v  # dvdt
g = ( - ( alpha + gamma ) + rho * v ) * u                 # dudt

# ヌルクラインと固定点
solutions = sp.solve( [ f , g ] , ( v , u ) )
print( solutions )

# v = 0 や u = 0 を含む解は経済的に意味がないので除く
fix_v , fix_u = [ sp.simplify( c ) for c in [ s for s in solutions if s[0] != 0 and s[1] !=
        0 ][0] ]
print( fix_v , fix_u )
print( fix_v.subs( param_val ) , fix_u.subs( param_val ) )

# ヤコビ行列
J_sym = sp.Matrix( [ [ f.diff( v ) , f.diff( u ) ] , [ g.diff( v ) , g.diff( u ) ] ] )
J_fix = sp.simplify( J_sym.subs( [ ( v , fix_v ) , ( u , fix_u ) ] ) )  # 固定点での
        J（記号のまま）
J_num = J_fix.subs( param_val )

print( J_sym )
print( J_fix )
print( J_num )
print( J_num.eigenvals() )
```

実行結果

```
[(0, 0), (-(alpha + gamma)*(-alpha*sigma - beta*sigma + 1)/(rho*(alpha*sigma + beta*sigma -
        1)), -alpha*sigma - beta*sigma + 1)]
(alpha + gamma)/rho -alpha*sigma - beta*sigma + 1
0.0600000000000000 0.940000000000000
Matrix([[-alpha - beta - u/sigma + 1/sigma, -v/sigma], [rho*u, -alpha - gamma + rho*v]])
Matrix([[0, (-alpha - gamma)/(rho*sigma)], [rho*(-alpha*sigma - beta*sigma + 1), 0]])
Matrix([[0, -0.0300000000000000], [0.470000000000000, 0]])
{-0.118743420870379*I: 1, 0.118743420870379*I: 1}
```

考察

1. ヌルクラインと固定点

dv/dt = 0となるのは、v=0またはu=1 - $\sigma(\alpha + \beta)$=0.94のとき。また、du/dt = 0となるのは、u=0またはv=($\alpha + \gamma$)/ρ = 0.06の時である。
固定点は(0,0)および(v,u)=(($\alpha+\gamma$)/ρ, 1 − $\sigma(\alpha+\beta)$) = (0.06, 0.94)の二つである。
この時、v=0やu=0は、経済的に考えて意味がないため、前者は無視できる。

2. 相平面とベクトル場から読み取れること

図を見ると、初期値(0.06, 0.92)から出た軌跡は、固定点を囲んでいる閉曲線であり、同じ道を何度も辿っている。vの範囲は0.055〜0.065、uが0.92〜0.96である。
横軸=v、縦軸=uとした時、これらは反時計回りに回る。なぜなら、雇用率が高いv > 0.06ではuが増加し、分配率が高いu > 0.94ではvが減少するためである。
そのため、経済的に見れば雇用率が上がる->賃金および労働分配率が上がる->利潤が減る->投資が減る->雇用率が下がるというサイクルが見える。これが繰り返されることにより景気循環が生まれると考えられる。

3. ヤコビ行列と固有値

固定点におけるヤコビ行列は、J = [[0, −($\alpha+\gamma$)/($\rho\sigma$)], [ρ(1 − $\sigma(\alpha+\beta)$), 0]]である。
ここに数値を代入すると、J = [[0, −0.03], [0.47, 0]]であり、固有値はλ=±0.1187iであり、純虚数であると言える。

4. 固有値による安定性解析

固有値は純虚数であり、実部は0である。そのため、固定点はセンターであり、軌跡は固定点に近づくこともなく、遠ざかることもない。また、漸近安定でも不安定でもない中立安定であると言える。
周期は約2π/0.1187 ≒ 53であり、時系列の約10周期/500とも一致する。
Sel'kovモデルと比べると、
Sel'kov（a=0.06）の固有値は実部が正の複素数で、不安定スパイラルである。軌跡はどの初期値から出ても1本の閉曲線に引き寄せられる。
一方でGoodwinでは、閉曲線は初期値ごとに違う。初期値を変えることで別の大きさの円となり、隣の軌道も閉曲線である。